TEE Private Intent Network: Technical Spec

Nov 6, 2026 · @Apurva

The MVP needs no custom privacy smart contract. The work is a TypeScript enclave service, a Next.js app and one CRE workflow.

1. Smart contracts: what is needed and what is not

No custom contract is required for the core flow. Custody accounts are plain EOAs, and the ledger lives inside the enclave. That is the point of the design, and it also means Solidity work is close to zero. The real work is the backend.

Contract	Needed?	Purpose
Custom privacy-pool contract	No, by design	Removing it is the whole idea
Uniswap Universal Router, Permit2, WETH, pool	External, already deployed	Swap execution. Take addresses from the Uniswap docs for your chosen chain
Test ERC-20 (for example MockUSDC)	Only if the chosen network has no liquid pair	Not needed on a mainnet fork
Chainlink ETH/USD data feed	External, optional	Second price source for the enclave's cross-check
EnclaveRegistry	Optional	Publishes the enclave public key and build hash so anyone can look up which enclave is current. Holds no user data
Operator staking and slashing	No, out of scope for v1	Multi-node work, see your brief
Paymaster or account abstraction	No	Custody EOAs pay their own gas

Optional EnclaveRegistry sketch (unaudited, about 15 lines)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract EnclaveRegistry {
    struct Entry { bytes32 buildHash; bytes pubKey; uint64 publishedAt; }
    address public immutable operator;
    Entry public current;
    event EnclavePublished(bytes32 indexed buildHash, bytes pubKey);

    constructor() { operator = msg.sender; }

    function publish(bytes32 buildHash, bytes calldata pubKey) external {
        require(msg.sender == operator, "not operator");
        current = Entry(buildHash, pubKey, uint64(block.timestamp));
        emit EnclavePublished(buildHash, pubKey);
    }
}
```

It cannot verify a TEE quote on-chain. It is only a public pointer, so skip it if time is short.

Contracts the executor must handle: a swap that starts from ETH needs a wrap step. A swap that starts from an ERC-20 needs an approval, through Permit2 if you use the Universal Router. Confirm the exact flow in the Uniswap docs before coding the executor.

If you want a contract-heavy project: this design is not it. Prizes that expect your own contracts, such as 1inch Aqua or ENSv2, would need a different build.

2. Repo layout

One TypeScript monorepo, so the frontend, the enclave and the CRE workflow share the same message types. Each person from the plan owns one folder.

```
private-intent-network/
  apps/
    web/                    # Next.js frontend (person D)
      app/                  # routes: deposit, swap, withdraw, status
      components/
      lib/                  # envelope encryption, session key, api client
  services/
    enclave/                # the TEE service
      src/
        server.ts           # Fastify routes, body limits, validation
```

```
        envelope.ts           # decrypt incoming requests (person A)
        keys.ts               # HD seed, derive EOAs, signing guard (person
A)
        ledger.ts             # balances, reserve, release, consume (person
B)
        journal.ts            # append-only hash-chained event log (person B)
        auth.ts               # EIP-712 verify, nonces, expiry (person B)
        intents.ts            # intent store and condition check (person B)
        watcher.ts            # deposit and receipt tracking (person C)
        inventory.ts          # per-EOA balances, EOA selection (person C)
        executor.ts           # swap, payout, tx lifecycle (person C)
        oracle.ts             # price reports and cross-check (person C)
        reconcile.ts          # invariant check
      Dockerfile
  packages/
    shared/                   # types, EIP-712 definitions, constants
  workflows/
    price-report/             # Chainlink CRE workflow (person D)
  contracts/                  # optional EnclaveRegistry only (Foundry)
  scripts/
    seed-inventory.ts         # fund and label the custody EOAs
    demo-fixture.ts           # two users, scripted run
  test/                       # Vitest suites and fork tests
  FEEDBACK.md                 # required by the Uniswap prize
  README.md
  LICENSE
```

Tooling: pnpm workspaces, TypeScript strict mode, viem, zod, Fastify, Vitest, fast-check for property tests, Foundry and Anvil for the fork.

3. Shared types and EIP-712 messages

These live in `packages/shared` so the browser signs exactly what the enclave verifies. Price is a fixed-point number: tokenOut per 1 tokenIn in human units, multiplied by 1e18, so token decimals never leak into comparisons.

```
// packages/shared/src/eip712.ts
import type { Address, Hex } from 'viem';

export const NATIVE: Address = '0x0000000000000000000000000000000000000000';

export const domain = (chainId: number, enclaveKeyHash: Hex) =>
```

```
  ({ name: 'PrivateIntentNetwork', version: '1', chainId, salt:
enclaveKeyHash }) as const;

export const types = {
  SessionGrant: [
    { name: 'owner', type: 'address' },
    { name: 'sessionKey', type: 'address' },
    { name: 'expiry', type: 'uint64' },
  ],
  DepositRequest: [
    { name: 'owner', type: 'address' },
    { name: 'asset', type: 'address' },
    { name: 'nonce', type: 'uint64' },
    { name: 'expiry', type: 'uint64' },
  ],
  Intent: [
    { name: 'owner', type: 'address' },
    { name: 'tokenIn', type: 'address' },
    { name: 'tokenOut', type: 'address' },
    { name: 'amountIn', type: 'uint256' },
    { name: 'comparator', type: 'uint8' },     // 0: price >= trigger, 1:
price <= trigger
    { name: 'triggerPrice', type: 'uint256' }, // 1e18 fixed point
    { name: 'maxSlippageBps', type: 'uint16' },
    { name: 'nonce', type: 'uint64' },
    { name: 'expiry', type: 'uint64' },
  ],
  Withdrawal: [
    { name: 'owner', type: 'address' },
    { name: 'token', type: 'address' },
    { name: 'amount', type: 'uint256' },
    { name: 'to', type: 'address' },
    { name: 'nonce', type: 'uint64' },
    { name: 'expiry', type: 'uint64' },
  ],
} as const;
```

```
// packages/shared/src/api.ts
export type Envelope = { enc: string; ct: string };  // HPKE-sealed JSON,
base64

export type SignedRequest<T> = { message: T; signature: Hex };

export type Balance = {
  token: Address;
  available: string;   // base units as decimal strings
  reserved: string;
  pending: string;
};

export type IntentStatus =
  | 'pending' | 'executing' | 'done' | 'failed' | 'cancelled' | 'expired';
```

Rules the enclave applies to every signed message

- Recover the signer from the signature and require it to equal `owner` (or the granted session key for reads).
- Require `expiry` in the future and at most 5 minutes ahead, with at most 60 seconds of clock skew.
- Require `nonce` greater than the owner's last accepted nonce, checked and written inside the serialized ledger queue.
- Reject a domain `chainId` or `salt` that does not match.

Session grants (optional, saves wallet popups): the user signs one SessionGrant for a browser-held key valid for about 30 minutes. Balance and status reads are signed by that key. Deposits, intents and withdrawals always need a fresh wallet signature.

4. Enclave backend: modules and interfaces

Every state change goes through one serialized queue and is written to the journal before it takes effect. State in memory is always rebuildable by replaying the journal.

```
// services/enclave/src/queue.ts
export interface SerialQueue {
  run<T>(op: () => Promise<T>): Promise<T>;   // one op at a time, in arrival
order
}
```

```
// journal.ts: append-only, each entry carries the hash of the previous one
export type JournalEvent =
  | { t: 'NONCE_USED'; owner: Address; nonce: bigint }
  | { t: 'DEPOSIT_SESSION_OPENED'; id: string; owner: Address; eoa: Address;
expiry: number }
  | { t: 'DEPOSIT_CREDITED'; id: string; amount: bigint; txHash: Hex }
  | { t: 'INTENT_CREATED'; id: string; owner: Address; intent: IntentMsg }
  | { t: 'EXEC_BROADCAST'; opId: string; eoa: Address; nonce: number; txHash:
Hex; raw: Hex }
  | { t: 'EXEC_SETTLED'; opId: string; amountOut: bigint }
  | { t: 'EXEC_FAILED'; opId: string; reason: string }
  | { t: 'WITHDRAWAL_LOCKED'; id: string; owner: Address; token: Address;
amount: bigint; to: Address }
  | { t: 'WITHDRAWAL_BROADCAST'; id: string; eoa: Address; nonce: number;
txHash: Hex; raw: Hex }
  | { t: 'WITHDRAWAL_CONFIRMED'; id: string }
  | { t: 'WITHDRAWAL_FAILED'; id: string; reason: string };

export interface Journal {
  append(e: JournalEvent): Promise<void>;   // encrypted at rest, fsync
before returning
  replay(): AsyncIterable<JournalEvent>;
}
```

```
// ledger.ts: per owner and token, all amounts bigint
export interface Ledger {
  credit(owner: Address, token: Address, amt: bigint, ref: string): void;
  reserve(owner: Address, token: Address, amt: bigint, ref: string): void;
// throws InsufficientBalance
  release(ref: string): void;                       // reserved back to
available
  consume(ref: string): void;                       // reserved leaves the
system
  balances(owner: Address): Balance[];
  totals(): Map<Address, { available: bigint; reserved: bigint }>;  // for
reconcile
}

// keys.ts: the only code that touches private keys
export interface KeyManager {
  init(): Promise<void>;                            // load sealed seed, or
generate inside the enclave
  allocate(role: 'deposit' | 'inventory'): CustodyEOA;
  list(): CustodyEOA[];
  sign(eoa: Address, tx: TxRequest, opId: string): Promise<Hex>;
}
```

Signing guard: before an operation starts, the executor registers `opId` with the expected destination, value and calldata hash. `KeyManager.sign` refuses any transaction that does not match a registered, journaled operation, and each opId signs once. There is no endpoint that returns a key or a seed.

```
// intents.ts
export interface IntentStore {
  add(owner: Address, msg: IntentMsg): string;                    //
reserves amountIn via Ledger
  due(price: PriceReport): StoredIntent[];                         //
conditions met and not expired
  mark(id: string, s: IntentStatus): void;
}

// inventory.ts: physical balances per custody EOA
export interface Inventory {
  refresh(): Promise<void>;                                        // read
balances from chain
  pick(token: Address, amt: bigint, exclude: Set<Address>): CustodyEOA |
```

```
null;
  hold(eoa: Address, token: Address, amt: bigint, opId: string): void;
  free(opId: string): void;
}

// executor.ts
export interface Executor {
  runIntent(i: StoredIntent): Promise<void>;                          // quote,
sign, broadcast, settle
  runWithdrawal(w: LockedWithdrawal): Promise<void>;                  //
transfer, confirm, finalize
  recover(): Promise<void>;                                           // on
boot: settle or fail unfinished ops
}

// oracle.ts
export type PriceReport = { pair: string; price: bigint; ts: number; source:
string };  // 1e18 fixed point
export interface Oracle {
  accept(r: PriceReport, authHeader: string): Promise<PriceReport>;   //
auth, staleness, cross-check
}

// reconcile.ts
export function reconcile(): { ok: boolean; perToken: Array<{ token: Address;
onChain: bigint; claims: bigint }> };
```

Boot order: init keys, replay the journal into the ledger and intent store, refresh inventory, run `Executor.recover()`, then open the server and start the watcher.

5. API reference

All POST bodies are an `Envelope`: the inner JSON is sealed to the enclave public key with HPKE (pick one library, for example `@hpke/core`, and verify it in your build). Amounts are decimal strings in base units. Errors return `{ error: code }` with no detail about other users.

Method and path	Inner request	Response
GET /attestation	none	`{ quote, enclavePubKey, buildHash, chainId }`

Method and path	Inner request	Response
POST /session	`SessionGrant` + wallet signature	`{ ok: true, expiresAt }`
POST /deposit-session	`DepositRequest` + signature	`{ sessionId, depositAddress, expiresAt }`
POST /balance	session-signed query	`{ balances: Balance[] }`
POST /intents	`Intent` + signature	`{ intentId, status: 'pending' }`
POST /intents/status	session-signed query	`{ intents: [{ id, status, amountOut? }] }`
POST /intents/cancel	`{ intentId }` + signature	`{ status: 'cancelled' }`
POST /withdrawals	`Withdrawal` + signature	`{ withdrawalId, status: 'locked' }`
POST /oracle/price	`PriceReport`, plus an auth header	`{ accepted: true }`
GET /health/reconcile	none	`{ ok, perToken: [{ token, onChain, claims }] }`

Example: create an intent (inner JSON before sealing)

```
{
  "message": {
    "owner": "0xAlice...",
    "tokenIn": "0x0000000000000000000000000000000000000000",
    "tokenOut": "0xUSDC...",
    "amountIn": "100000000000000000",
    "comparator": 0,
    "triggerPrice": "3000000000000000000000",
    "maxSlippageBps": 50,
    "nonce": "7",
    "expiry": "1790000000"
  },
  "signature": "0x..."
}
```

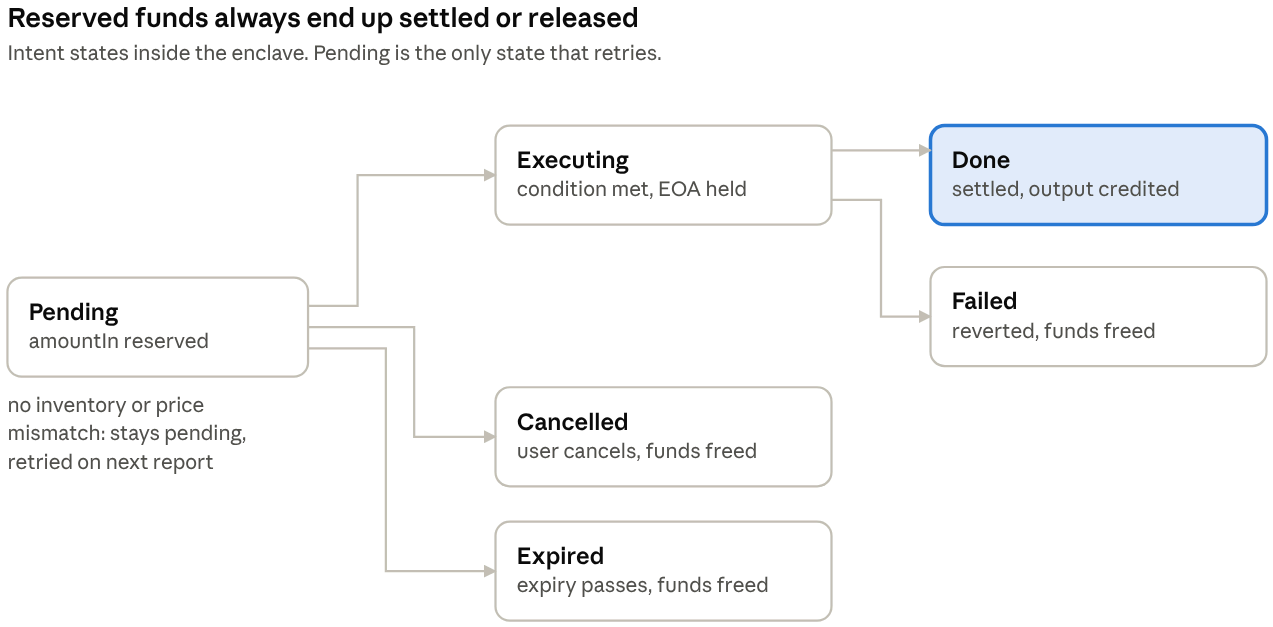
This reads as: swap 0.1 ETH for USDC once ETH is at or above 3,000 USDC, accept at most 0.5% slippage.

Error codes: `BAD_SIGNATURE`, `EXPIRED`, `NONCE_USED`, `WRONG_CHAIN`, `INSUFFICIENT_BALANCE`, `NO_INVENTORY`, `UNKNOWN_SESSION`, `STALE_PRICE`, `PRICE_MISMATCH`.

Server hardening: body size limit (for example 16 KB), zod validation on the decrypted JSON, a per-IP rate limit, CORS limited to your frontend origin, and structured logs that never contain addresses, amounts or intent contents.

6. Chain logic

Every on-chain operation follows one order: reserve funds, journal, sign, broadcast, confirm, settle. Nothing is signed before its journal entry exists.



intent lifecycle · 6 states

Only a confirmed swap consumes the reserved funds. Every other ending gives them back to the owner's available balance.

Deposits (`watcher.ts`)

1. When a session opens, record the assigned EOA's native balance as its baseline.

2. On each new block, read that balance at `head - CONFIRMATIONS`. Credit the owner with the increase over what is already credited. Journal `DEPOSIT_CREDITED` first, then call `ledger.credit`.
3. Crediting by the recipient's balance, not by the sender, also works for smart-contract wallets. The sender is recorded but not required. This replaces the sender match in section 5 of the plan.
4. After expiry and one last check, close the session and relabel the EOA as inventory. Later transfers to a closed address are not credited and need a manual refund.

Inventory selection (`inventory.ts`)

```
function pick(token: Address, amt: bigint, exclude: Set<Address>): CustodyEOA
| null {
  const need = token === NATIVE ? amt + GAS_RESERVE_WEI : amt;
  return eoas
    .filter(e => e.role === 'inventory' && !e.inFlight &&
!exclude.has(e.address))
    .filter(e => freeBalance(e, token) >= need)      // physical balance
minus holds
    .sort((a, b) => a.lastUsedAt - b.lastUsedAt)      // least recently used
first
    [0] ?? null;
}
```

`exclude` holds the EOAs that received this owner's deposits and ran this owner's earlier swaps. This is what separates the funded address from the executing address. With 4 to 6 EOAs the choice is small, so say that in the demo.

Swap execution (`executor.ts`)

1. Create an `opId`, call `inventory.hold`, and register the expected destination with the signing guard.
2. Get a quote for exactly `amountIn` from the Uniswap Trading API, with the chosen EOA as recipient. Set `minOut = quote.out * (1 - maxSlippageBps / 10000)`.
3. Compare the quote's implied price with the oracle price. If they differ by more than `PRICE_TOLERANCE_BPS`, stop and leave the intent pending.
4. Treat the API's transaction as untrusted data. Require `to === UNIVERSAL_ROUTER_ADDRESS`, run it through `eth_call`, and require the simulated output to be at least `minOut`.

5. Take the next nonce from a local per-EOA counter, sign, compute the hash from the serialized transaction, journal `EXEC_BROADCAST`, then send the raw transaction.
6. After `CONFIRMATIONS` blocks, sum the ERC-20 Transfer logs to the EOA in the receipt. That sum is `amountOut`.
7. Journal `EXEC_SETTLED`, then `ledger.consume` the reserved input and credit the owner's output minus the fee. Free the hold. Book the gas used to the fee ledger.

If the receipt shows a revert, journal `EXEC_FAILED` and release the reservation. Gas spent is the pool's cost.

Withdrawal

1. `ledger.reserve`, journal `WITHDRAWAL_LOCKED`, then `inventory.pick` with the owner's EOAs excluded.
2. Build a native send, or `transfer(to, amount)` on the token contract. The destination is the signed `to`.
3. Sign, journal `WITHDRAWAL_BROADCAST`, send, wait for confirmations, then `ledger.consume` and journal `WITHDRAWAL_CONFIRMED`.
4. If `pick` returns nothing, release the lock and return `NO_INVENTORY`.

Gas: every inventory EOA keeps `GAS_RESERVE_WEI`, which reconcile excludes from claims. Top-ups from one funder link the EOAs publicly. List that as a known leak.

Recovery on boot (`executor.recover`): for each broadcast without a terminal event, look up the receipt by hash. If found, settle or fail it. If not found and the EOA nonce has not moved, rebroadcast the identical raw transaction. If the nonce moved, mark the operation failed for manual review. For this, store the raw signed transaction in the BROADCAST journal events.

Known gaps: a stuck transaction is not fee-bumped in the MVP, and a reorg deeper than `CONFIRMATIONS` after settlement is not handled.

7. Frontend

A Next.js app that never shows a private key and never needs a custom wallet. The user's existing wallet signs; the app seals the request and sends it.

Route	What it does	Calls
`/`	Connect wallet, show the attestation badge (build hash, enclave key, link to the reproducible build)	`GET /attestation`
`/deposit`	Sign a DepositRequest, show the assigned address and countdown, then send ETH from the wallet	`POST /deposit-session`
`/swap`	Form for token pair, amount, trigger price, comparator, slippage, expiry. List of intents with status	`POST /intents`, `/intents/status`, `/intents/cancel`
`/withdraw`	Choose token, amount and destination, sign, show status and the payout transaction link	`POST /withdrawals`
`/balance` (panel on every page)	Available, reserved and pending per token	`POST /balance`

Request flow in `lib/`

1. Fetch `/attestation` once and keep the enclave key. Check that its hash equals the EIP-712 `salt` the app will sign with.
2. Build the typed message from `packages/shared`, ask the wallet to sign it with viem's `signTypedData`, and show the user a readable summary first.
3. Seal `{ message, signature }` to the enclave key with HPKE and POST the envelope.
4. Poll status every 5 seconds while an operation is open, using the session key if one is granted.

Anon-RPC: route chain reads (wallet balance, transaction receipts, token reads) through the Anon-RPC browser harness instead of a normal RPC provider. Follow the wallet and app guide on the Anon-RPC site; I have not checked how the harness plugs into viem. Anything you leave out, such as the wallet's own signing RPC, goes in the README leak list.

UX rules

- Show amounts in human units but sign base units. Display the exact trigger price the wallet will sign.
- Never display custody EOA addresses other than the one deposit address for the open session.
- Show "TEE-simulated" in a banner whenever the attestation badge is not real.
- No analytics or third-party scripts. The EF CROPS prize rejects trackers in the client.

Stack: Next.js, TypeScript, wagmi, viem, RainbowKit, TanStack Query, Tailwind. Use `packages/shared` for every message type.

8. CRE workflow

The workflow is a scheduled messenger: every minute it fetches a price and posts it to the enclave. Intent contents never reach it, because it only handles prices.

Setup, from the CRE overview: create an account on the CRE platform, install the CRE CLI, and build the workflow with the TypeScript SDK. You can simulate without approval. Deploying to a live DON is Early Access and needs approval, which you do not need for the prize.

Files (scaffold them with the CLI, then check the layout it generates)

```
workflows/price-report/
  main.ts           # trigger and callback
  workflow.yaml     # workflow settings
  config.json       # pair, price API URL, enclave URL, tolerance
```

Logic sketch (helper names are mine; use the imports the scaffold gives you)

```
handler(
  cronTrigger.trigger({ schedule: '0 * * * * *' }),   // every minute (six-
field cron, seconds first)
  onCronTrigger,
);

function onCronTrigger(runtime: Runtime<Config>) {
  const price = fetchPrice(runtime);                   // HTTP capability:
price API, parsed to 1e18 fixed point
  const report = { pair: runtime.config.pair, price: price.toString(), ts:
nowSeconds(), source: 'cre' };
  postToEnclave(runtime, report);                     // HTTP capability: POST
/oracle/price with auth header
  return {};
}
```

Optional second source: read the Chainlink ETH/USD data feed on the chosen chain and take the median of the two.

Authentication. Keep a shared secret in CRE secrets and send `X-Report-Auth: HMAC_SHA256(secret, body)`. The enclave checks the HMAC in constant time, requires `ts` within 90 seconds, and rejects any `ts` not newer than the last accepted one. A shared secret means CRE is trusted for freshness, not for the price itself.

Why that is safe enough: the enclave also compares every report with its own Uniswap quote (`PRICE_TOLERANCE_BPS`) and enforces `minOut` on the swap, so a wrong report cannot drain funds.

Two things to verify in the CRE docs before building

- Whether an HTTP POST inside a workflow runs once or on several DON nodes. Make `/oracle/price` idempotent anyway: key each report by `(pair, floor(ts / 60))` and ignore repeats.
- Whether the enclave can verify a DON-signed report instead of using the HMAC. I have not checked how; treat it as a stretch.

Demo setup: simulation runs on your machine and makes real calls to live APIs and public EVM chains, so the enclave must be reachable by URL, for example through a tunnel when it runs locally. Show the simulation output and the enclave log line that accepted the report. The CLI steps are in the simulating workflows guide.

9. Deployment, configuration and attestation

The enclave ships as one Docker image built reproducibly, so a judge can rebuild it and compare hashes. How a quote is fetched and how secrets reach the enclave differs per TEE provider, so treat those steps as *verify* items for whichever platform you pick.

```
FROM node:20-bookworm-slim@sha256:<pin-the-digest> AS build
WORKDIR /app
COPY pnpm-lock.yaml package.json pnpm-workspace.yaml ./
COPY packages ./packages
COPY services/enclave ./services/enclave
RUN corepack enable && pnpm install --frozen-lockfile && pnpm --filter
enclave build

FROM node:20-bookworm-slim@sha256:<pin-the-digest>
WORKDIR /app
COPY --from=build /app/services/enclave/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
CMD ["node", "dist/server.js"]
```

Configuration

Variable	Example	Purpose
`CHAIN_ID`	`11155111`	Execution network and EIP-712 domain
`RPC_URL`	provider URL	Chain reads and broadcasts. Injected as a secret
`CONFIRMATIONS`	`2`	Blocks before crediting or settling
`GAS_RESERVE_WEI`	`10000000000000000`	Kept in each inventory EOA, excluded from claims
`UNIVERSAL_ROUTER_ADDRESS`	from Uniswap docs	Pinned swap destination
`WETH_ADDRESS`, `TOKEN_OUT_ADDRESS`	from Uniswap docs	Pair for the demo
`UNISWAP_API_KEY`	from the developer platform	Quotes and swap calldata

Variable	Example	Purpose
`PRICE_TOLERANCE_BPS`	`100`	Max gap between a report and the enclave's own quote
`MAX_SLIPPAGE_BPS_CAP`	`100`	Upper limit on a user's slippage setting
`FEE_BPS`	`10`	Fee on the input amount (simpler than a flat fee; pick one)
`DEPOSIT_SESSION_TTL_SEC`	`900`	Lifetime of a deposit address
`ORACLE_HMAC_SECRET`	random 32 bytes	Authenticates price reports
`ADMIN_ADDRESS`, `SEEDING_ENABLED`	wallet, `true` then `false`	Operator seeding endpoint, turned off after setup
`MAX_AMOUNT_NATIVE_WEI`	demo cap	Limits exposure during the demo
`SIM_MODE`	`false`	Shows the TEE-simulated banner when `true`
`ALLOWED_ORIGIN`	frontend URL	CORS

Boot and attestation

1. At boot the enclave generates its HPKE key pair and loads or creates the sealed HD seed.
2. It requests a platform quote whose report data commits to `sha256(enclavePubKey || buildHash)`.
3. `GET /attestation` returns the quote, the public key, the build hash and the chain ID. The frontend uses the key hash as the EIP-712 `salt`.
4. The journal is encrypted with a key derived inside the TEE. If the platform has no sealed storage, use a persistent volume with a platform-derived key. Only `SIM_MODE` may fall back to a key from an environment variable.

Seeding: a signed admin call returns the inventory EOA addresses once. `scripts/seed-inventory.ts` funds them with ETH and the output token, then you set `SEEDING_ENABLED=false`. These addresses become public the moment they are funded, which is a known leak.

Release steps

1. Build the image, record its digest in the README.
2. Deploy to the TEE provider and fetch `/attestation`.
3. Verify the quote with a CLI script and check that its report data matches the public key.
4. Seed the inventory, then disable seeding.
5. Point the frontend and the CRE workflow at the enclave URL.

10. Tests and security checklist

Chain tests run against an Anvil fork. Set `CHAIN_ID` to whatever the fork reports, because it must match the EIP-712 domain. Run `reconcile()` at the end of every suite.

Suite	Covers	Must assert
`ledger.property.test.ts`	Random sequences of credit, reserve, release, consume (fast-check)	Never negative; totals conserved
`auth.test.ts`	Signatures, expiry, nonces, wrong chain or salt, session grants	Each bad input returns its error code
`concurrency.test.ts`	Two withdrawals of 80% at once; intent versus withdrawal race	Exactly one wins
`journal.test.ts`	Replay rebuilds state; one altered entry	Same balances; hash chain breaks on tamper
`recovery.test.ts`	Kill after `EXEC_BROADCAST`, before settlement	Reboot settles once, never executes twice
`watcher.test.ts`	Deposit confirmations, repeat checks, closed sessions	Credited once, only after N blocks
`executor.fork.test.ts`	Swap, `minOut`, malicious calldata, revert	Swap runs from a non-depositor EOA; `to` other than the router is refused; revert releases funds

Suite	Covers	Must assert
`oracle.test.ts`	Bad HMAC, stale and repeated `ts`, price outside tolerance	Report rejected or intent held
`e2e.fork.test.ts`	The full demo fixture with two users	Reconcile is `ok` at the end

Security checklist

- [ ] No endpoint returns a key or seed. The only address list is the one-time admin seeding call (`GET /admin/eoas`, not in section 5), which is then disabled
- [ ] Signing guard rejects unregistered operations, and each `opId` signs once
- [ ] Router address pinned, and all API-provided calldata simulated before signing
- [ ] zod validation on every decrypted body, plus a body size limit
- [ ] HMAC compared in constant time
- [ ] Logs contain no addresses, amounts or intent fields
- [ ] CORS locked to the frontend, no third-party scripts in the client
- [ ] Demo caps on amounts and a pause switch
- [ ] Journal tamper test passes
- [ ] TEE-simulated banner shown whenever the attestation is not real
- [ ] README lists the known leaks: amounts and timing, the visible payout edge, gas-funding links between EOAs, the enclave's own RPC reads, a small anonymity set, and trust in the operator and hardware